Scout Streaming

SSE, pause, resume and stop — how Scout's answers travel, wait and end

A beginner-friendly guide to the streaming code in frontend-v3 (swo-3.0) and the Scout Core v2 service (rag), with diagrams and real code.

Edition: October 2026 · Code re-checked 29 Oct 2026 against branch ONLINE-5385_scout_release_2

Before you start

When a teacher asks Scout something, the answer appears word by word, like someone typing. Sometimes Scout stops halfway and waits — for the browser to do something, or for the teacher to choose. Sometimes the teacher presses **Stop**. This guide explains how all of that works, in simple steps, with the real code.

- **Chapters 1–3** explain SSE and how the server writes the stream.
- **Chapters 4–5** show how the browser reads it, and the states a turn can end in.
- **Chapters 6–8** are the heart of it: **pause**, **resume** and **stop**.
- **Chapter 9** lists the timers and what happens when something fails.

Every code block shows its file. Paths starting with `swo-3.0/` are the frontend repo; paths starting with `rag/` are the AI service. Snippets are shortened; `…` marks lines left out. The current reference is `swo-3.0/docs/scout/README.md` (section "One turn": the sequence and the full event list).

Contents

1. What SSE Is, and Why Scout Uses It 4

2. The Streaming Path in Scout 6

3. Server: Writing the Stream 8

4. Browser: Reading the Stream 12

5. How a Turn Can End 16

6. Pause: When Scout Must Wait 17

7. Resume: Carrying On After the Pause 22

8. Stop: Ending a Turn Early 26

9. Timers, Failures and Debugging 31

Chapter 1

What SSE Is, and Why Scout Uses It

One request, one long answer, sent piece by piece.

1.1 The idea in one paragraph

Normally a browser sends a request and waits until the whole answer is ready. A lesson pack can take more than a minute, so the teacher would stare at a spinner. With **Server-Sent Events (SSE)**, the server keeps the response open and writes small messages into it as soon as it has them: a status line, a few words, a card. The browser reads each message the moment it arrives.

> **In simple words**
> Think of live cricket commentary on the radio. You don't wait for the match to end to hear the result — you hear every ball as it happens. SSE is the radio; each event is one ball.

1.2 What it looks like on the wire

SSE is plain text with a very small format. Each message is an `event:` line, a `data:` line, and a blank line. A line that starts with `:` is a comment, which Scout uses as a heartbeat.

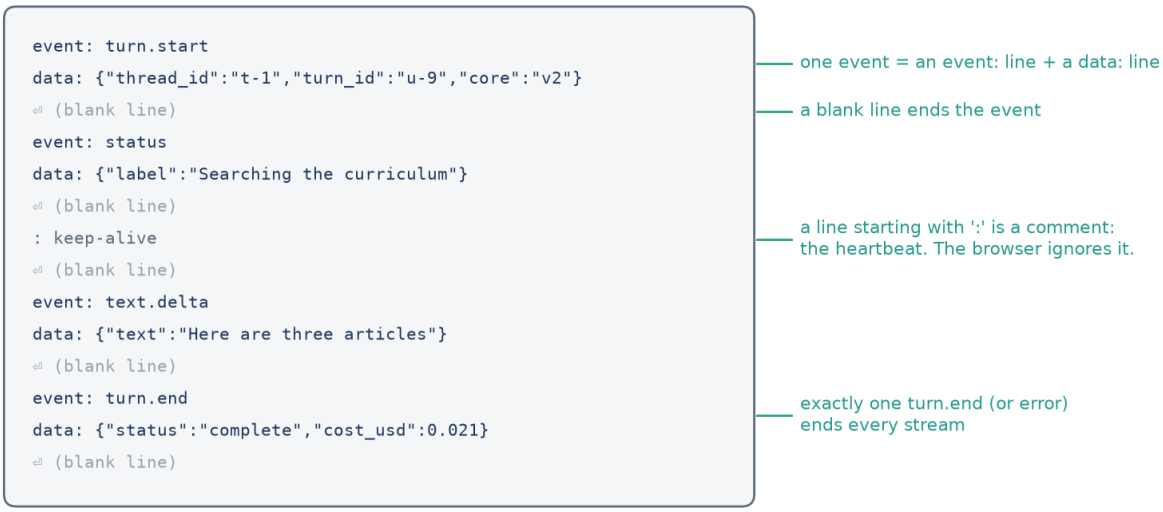


Figure 1.1 — Four Scout events on the wire, with a heartbeat in the middle.

1.3 Why SSE and not something else?

Option	Good at	Why not for Scout's chat
One normal request	Simple	The teacher waits for the whole answer, sometimes for minutes.
Polling (ask every second)	Simple servers	Many wasted requests; text still arrives in jumps.
WebSocket	Two-way, long-lived chat	Needs a connection that stays up between turns, which Lambda and API Gateway handle poorly. Scout only needs server → browser during a turn.
SSE over a POST	**One-way stream for one request**	**Fits a turn exactly: one request in, a stream of events out, then it closes.**

Why it is built this way

Scout does not use the browser's built-in `EventSource` object, because it can only send GET requests and cannot set an `Authorization` header. Scout uses `fetch()` with a POST body and reads the response stream itself (Chapter 4).

Chapter summary

- ✓ SSE keeps one HTTP response open and writes small text messages into it as they are ready.
- ✓ Each message is 'event:' + 'data:' + a blank line; a line starting with ':' is a comment (heartbeat).
- ✓ A Scout turn is one POST in and one stream of events out, so SSE fits better than polling or WebSockets.
- ✓ Scout reads the stream with fetch(), not EventSource, so it can POST and send the login token.

Chapter 2

The Streaming Path in Scout

From the agent loop in rag to words on the teacher's screen.

2.1 The whole path

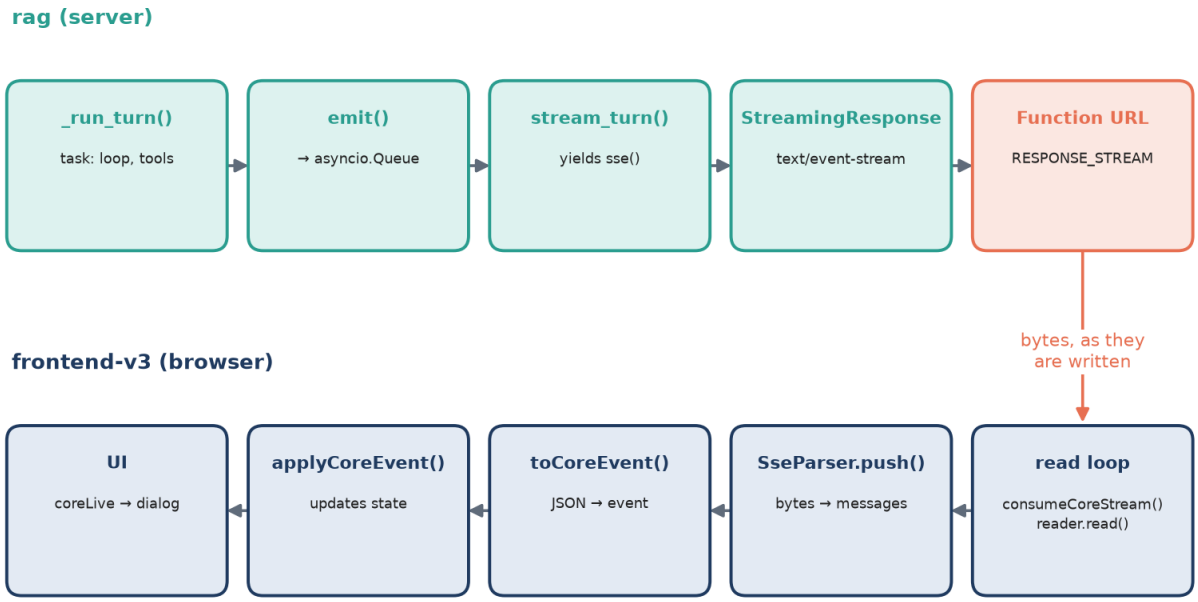


Figure 2.1 — Every piece the stream passes through, server above, browser below.

Piece	Job	File
`_run_turn()`	Runs the turn (loop, tools) and calls emit() for every event	`rag/…/core/runtime.py`
`stream_turn()`	Turns emitted events into SSE text, adds heartbeats and the time limit	`rag/…/core/runtime.py`
`StreamingResponse`	FastAPI sends each yielded string to the client straight away	`rag/…/routers/core_router.py`
Function URL	AWS passes the bytes through as they are written (RESPONSE_STREAM)	`rag/infra/swo/modules/scout-core-v2/`
`streamTurn()`	Sends the POST and checks the first response	`swo-3.0/…/ai/services/coreClient.ts`
`consumeCoreStream()`	Reads bytes, parses SSE, turns them into events, applies them to state	`swo-3.0/…/ai/services/coreStream.ts`
`coreLive`	What the Scout window paints while the answer arrives	`swo-3.0/…/ai/services/coreLiveTurn.ts`

2.2 The route that opens the stream

Everything that can go wrong **before** the model starts is answered as a normal JSON error. Only a turn that passed every check gets `200 text/event-stream`.

rag/lambdas/ai_assistant/src/routers/core_router.py — *shortened*

```
SSE_HEADERS = {
    "Cache-Control": "no-cache, no-transform",
    "X-Accel-Buffering": "no",
    "Connection": "keep-alive",
}
@router.post("/turn")
async def turn(request: ChatRequest, http_request: Request, host: Optional[str] = None):
    …                                    # name scrub, size checks, identity
    try:
        prepared = await preflight(request, user_id)
    except PreflightError as failed:
        return JSONResponse(status_code=failed.status, content=failed.body())
    …
    return StreamingResponse(stream_turn(request, user_id, prepared, started=started, auth=auth),
                             media_type="text/event-stream", headers=SSE_HEADERS)
```

Header	Why it is there
`Cache-Control: no-cache, no-transform`	No cache or proxy may store, compress or rewrite the stream.
`X-Accel-Buffering: no`	Tells nginx-style proxies not to hold bytes back until the response ends.
`Connection: keep-alive`	Keep the connection open while the turn runs.

Why it is built this way

API Gateway cannot stream and stops at 29 seconds. That is why chat turns go to the Scout Core v2 lambda through a **Function URL** with `invoke_mode = RESPONSE_STREAM`, and the Lambda Web Adapter runs this FastAPI app inside it. See the developer guide, Chapter 10.

Chapter summary

- ✓ Server path: _run_turn() → emit() → queue → stream_turn() → StreamingResponse → Function URL.
- ✓ Browser path: streamTurn() → consumeCoreStream() → SseParser → applyCoreEvent() → coreLive → screen.
- ✓ Early failures are plain JSON; only a checked turn gets 200 text/event-stream.
- ✓ SSE_HEADERS stop caches and proxies from buffering or changing the stream.

Chapter 3

Server: Writing the Stream

A producer, a queue and a consumer — plus a heartbeat and a time limit.

3.1 One event as text

rag/lambdas/ai_assistant/src/core/runtime.py

```
def sse(event: str, data: Dict[str, Any]) -> str:
    return f"event: {event}\ndata: {json.dumps(data, default=str, ensure_ascii=False)}\n\n"
```

That is the whole SSE format on the server: the event name, the JSON data on one line, and a blank line.

3.2 Producer and consumer

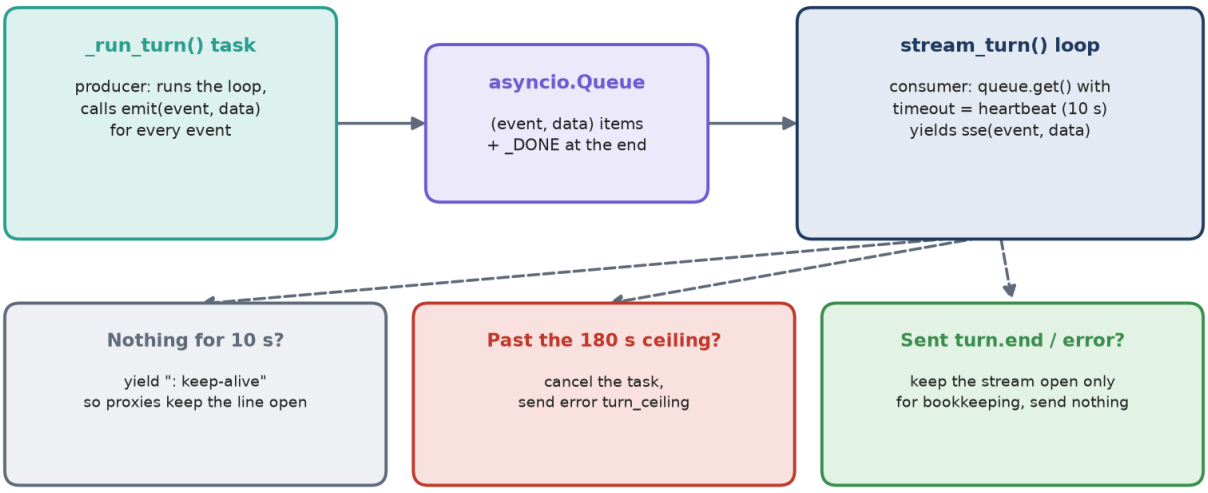


Figure 3.1 — The turn runs as a task that puts events in a queue; stream_turn() takes them out and writes them.

The turn itself runs as a separate asyncio **task**. Every time it has something to show, it calls `emit(event, data)`, which only puts the item in a queue. The `stream_turn()` generator waits on that queue and `yield`s each item as SSE text. FastAPI sends every yielded string to the browser immediately.

rag/lambdas/ai_assistant/src/core/runtime.py — *shortened*

```
async def stream_turn(request, user_id, prepared=None, started=None,
                      auth=None) -> AsyncIterator[str]:
    cfg = get_core_config()
    queue: asyncio.Queue = asyncio.Queue()
    cancel = CancelSignal()
    async def emit(event: str, data: Dict[str, Any]) -> None:
        await queue.put((event, data))
    remaining_ceiling = max(0.1, cfg.turn_ceiling_s - (time.monotonic() - started))
    task = asyncio.create_task(asyncio.wait_for(
        _run_turn(request, user_id, emit, started, prepared, cancel, auth=auth),
```

rag/lambdas/ai_assistant/src/core/runtime.py — *shortened*

```
        timeout=remaining_ceiling))
    task.add_done_callback(lambda _t: queue.put_nowait(_DONE))
    terminal_sent = False
    try:
        while True:
            remaining = cfg.turn_ceiling_s - (time.monotonic() - started)
            if remaining <= 0:
                task.cancel()
                if not terminal_sent:
                    yield sse("error", {"code": "turn_ceiling", "message": …, "retryable": True})
                    terminal_sent = True
                await asyncio.wait({task}, timeout=_CEILING_RECORD_GRACE_S)
                return
            try:
                item = await asyncio.wait_for(queue.get(), timeout=min(cfg.heartbeat_s, remaining))
            except asyncio.TimeoutError:
                if not terminal_sent:
                    yield ": keep-alive\n\n"
                continue
            if item is _DONE:
                if not terminal_sent:
                    yield sse("error", {"code": "internal", "message": …, "retryable": True})
                    terminal_sent = True
                return
            if terminal_sent:
                continue
            event, data = item
            yield sse(event, data)
            if event in _TERMINAL:                # ("turn.end", "error")
                terminal_sent = True
    finally:
        if not terminal_sent and not task.done():
            cancel.set("disconnect")
```

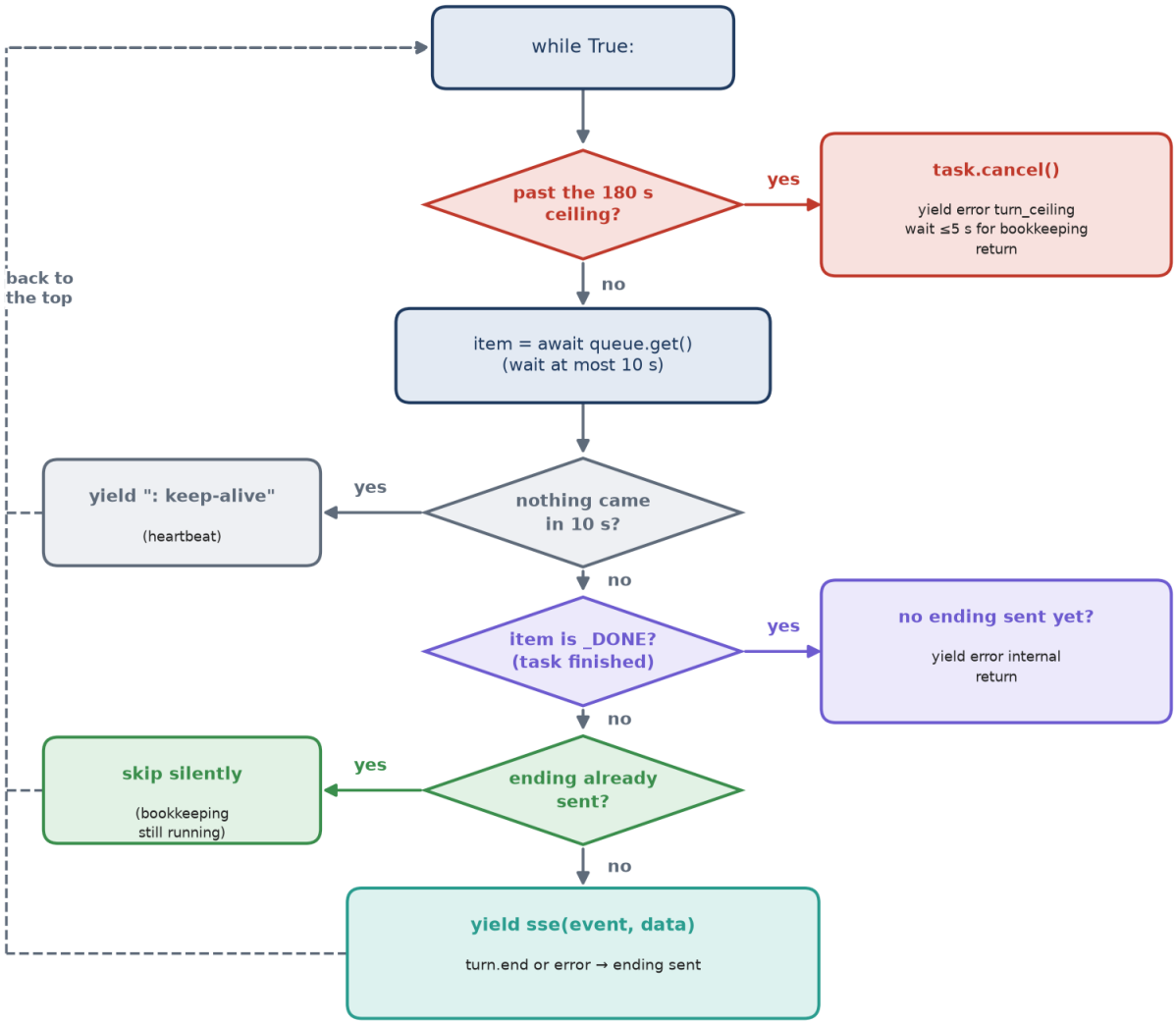


Figure 3.2 — One pass of the stream_turn() loop as a flowchart. Every branch either writes something, skips, or ends the stream.

Read it in five small parts:

- **Heartbeat:** if nothing arrives for `heartbeat_s` (10 s), it writes the comment `: keep-alive`. Proxies see traffic and keep the line open; the browser ignores it.
- **Ceiling:** past `turn_ceiling_s` (180 s) it cancels the task and sends an `error` event with code `turn_ceiling`.
- **Exactly one ending:** after `turn.end` or `error`, `terminal_sent` is true and nothing more is written.
- **Bookkeeping after the end:** the stream stays open (silently) until the task finishes saving the sidebar row, the title and telemetry, because on Lambda a closed stream freezes that work.
- **A task that dies** without an ending still produces one `error` event, so the browser never waits forever.

3.3 The events Scout sends

Event	Data	When
`turn.start`	thread_id, turn_id, model	First event, within about 300 ms
`status`	label	"Reading the article", "Searching the curriculum"
`text.delta`	text	The next piece of the reply
`artifact.start / .delta / .replace / .end`	artifact_id, kind, text, ref	A document streamed into a card
`content_results`	result cards	Search results
`adapted_content`	levelled article	After a leveling rewrite
`pending_action`	id, capability, args	The turn pauses for the browser to act (Chapter 6)
`pending_prompt`	id, question, options	The turn pauses to ask the teacher (Chapter 6)
`turn.end`	status, outcome, usage, cost_usd, timings	complete / paused / stopped
`error`	code, message, retryable	Something failed; a teacher-safe message

Here is how those events might be spread over one turn that runs a tool and then writes a document:



Figure 3.3 — One turn's events over time. While a tool works there is no text, only status labels and, after 10 s of silence, a heartbeat.

Chapter summary

- ✓ sse() writes 'event:', 'data:' and a blank line; that is the whole format.
- ✓ The turn runs as a task that emits into an asyncio.Queue; stream_turn() yields each item as SSE.
- ✓ A ': keep-alive' comment every 10 s of silence keeps proxies from closing the line.
- ✓ A 180 s ceiling ends a turn with error turn_ceiling; every stream ends with exactly one turn.end or error.

Chapter 4

Browser: Reading the Stream

fetch(), a reader loop, a small parser and a state object.

4.1 Sending the request

swo-3.0/frontend-v3/ai/services/coreClient.ts — streamTurn() — *shortened*

```
response = await fetch(`${coreRoot()}${coreTurnPath()}?${params.toString()}`, {
    method: 'POST',
    headers: {
        'Content-Type': 'application/json',
        Accept: 'text/event-stream',
        Authorization: auth.token,
    },
    body: requestBody,
    signal: controller.signal,
})
…
if (!response.ok || !response.body) {
    // Not a stream: a JSON error from the early checks. Show its message.
    …
    throw requestError(message, response.status, detail)
}
return await consumeCoreStream(response.body, { ...options, idleTimeoutMs: idleMs })
```

A timer also limits how long the browser waits for the first response headers (the same 60 s idle limit). There is no limit on the whole turn: only on **silence**.

4.2 The reader loop

`consumeCoreStream()` reads chunks of bytes. Each read races three things: the next chunk, the idle timer, and the abort signal (Stop).

swo-3.0/frontend-v3/ai/services/coreStream.ts — consumeCoreStream() — *shortened*

```
while (!isTurnOver(state)) {
    const idle = new Promise<'idle'>((resolve) => {
        timer = setTimeout(() => resolve('idle'), idleMs)
    })

    let result: ReadableStreamReadResult<Uint8Array> | 'idle' | 'aborted'
    try {
        result = await Promise.race([reader.read(), idle, aborted])
    } catch {
        if (signal?.aborted) throw abortError(state)     // Stop, not a network failure
        fail('network', CORE_STREAM_MESSAGES.network)
        break
    } finally {
        clearTimeout(timer)
    }
    if (result === 'aborted' || signal?.aborted) throw abortError(state)
    if (result === 'idle') { fail('idle_timeout', CORE_STREAM_MESSAGES.idle); break }
    if (result.done) {
        deliver(parser.push(decoder.decode()))
        deliver(parser.flush())
        if (!isTurnOver(state)) fail('stream_closed', CORE_STREAM_MESSAGES.closed)
        break
    }
    deliver(parser.push(decoder.decode(result.value, { stream: true })))
    if (parser.overflowed && !isTurnOver(state)) {
        fail('stream_overflow', CORE_STREAM_MESSAGES.overflow)
```

swo-3.0/frontend-v3/ai/services/coreStream.ts — consumeCoreStream() — *shortened*

```
            break
        }
    }
```

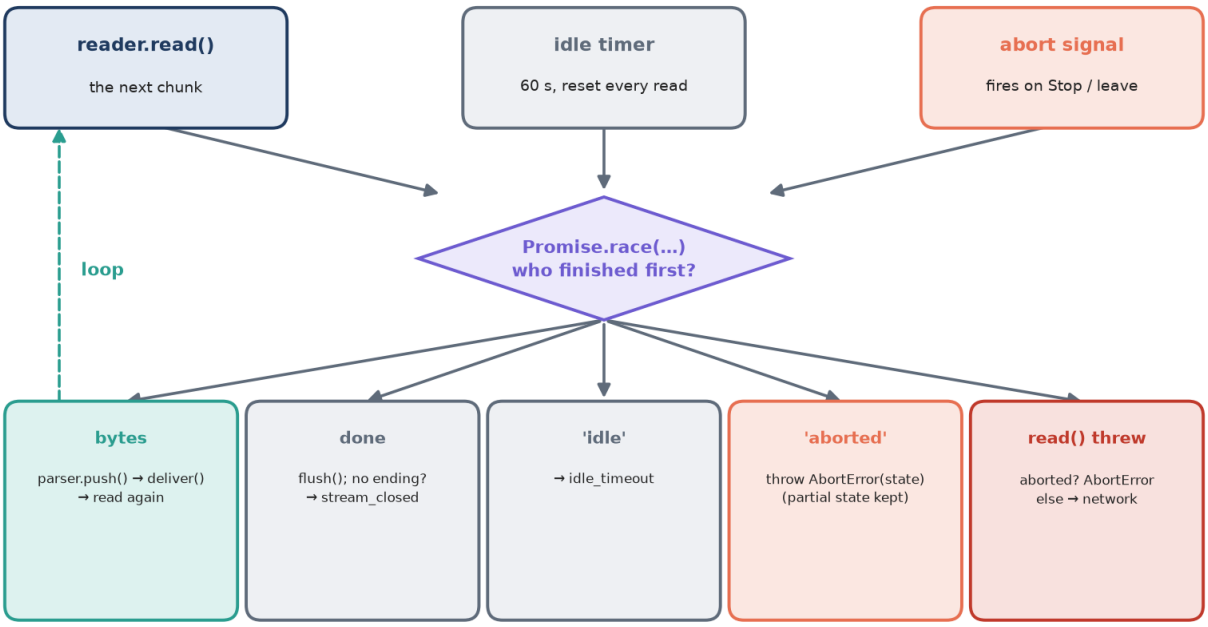


Figure 4.1 — Each pass of the reader loop: three things race, and five outcomes are possible.

In simple words
A chunk is not an event. One read may hold half an event, or three events. That is why the parser keeps the unfinished part and only hands out complete messages.

4.3 The parser

swo-3.0/frontend-v3/ai/services/coreStream.ts — SseParser — *shortened*

```
private line(line: string): SseMessage | null {
    if (line === '') return this.dispatch()        // blank line: the event is complete
    if (line.startsWith(':')) return null          // comment: the heartbeat, ignored
    const colon = line.indexOf(':')
    const field = colon < 0 ? line : line.slice(0, colon)
    let value = colon < 0 ? '' : line.slice(colon + 1)
    if (value.startsWith(' ')) value = value.slice(1)
    if (field === 'event') this.eventName = value
    else if (field === 'data') {
        this.dataLength += value.length + 1
        if (this.dataLength > SSE_MAX_BUFFER) { this.overflow(); return null }   // 1 MB guard
        this.dataLines.push(value)
    }
    return null
}
```

`push()` splits each chunk into lines (it handles `\n`, `\r\n` and a `\r` split across two chunks), and `line()` builds the event. `toCoreEvent()` then parses the JSON and checks its shape.

The picture below follows three real chunks through the parser. Notice how `"Sear"` and `"ching"` arrive in different chunks but leave as one event:

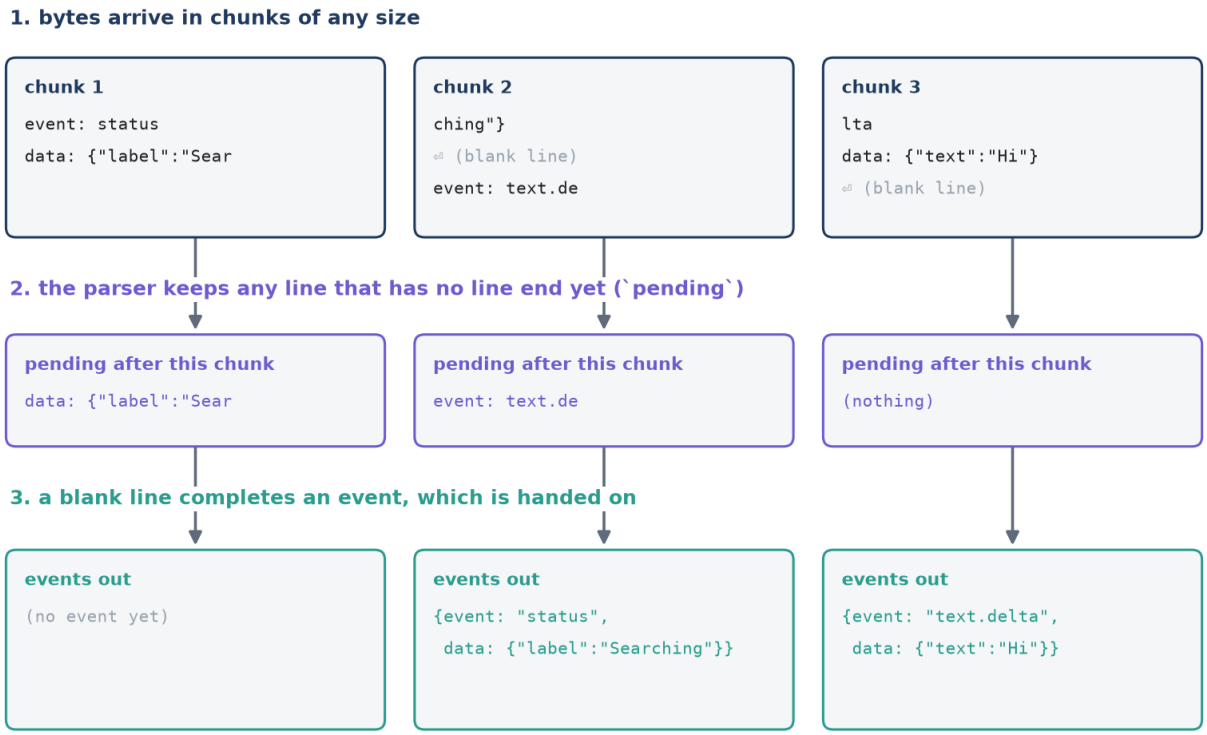


Figure 4.2 — Chunks in, complete events out. The parser holds any line that has no line end yet.

4.4 From events to the screen

swo-3.0/frontend-v3/ai/services/coreStream.ts — *shortened*

```
export function applyCoreEvent(state: CoreTurnState, event: CoreEvent): CoreTurnState {
    if (isTurnOver(state)) return state          // nothing after turn.end / error
    switch (event.type) {
        case 'turn.start':
            state.threadId = event.thread_id
            state.turnId = event.turn_id || null
            break
        case 'status':         state.status = event.label || null; break
        case 'text.delta':     state.text += event.text; break
        …
        case 'pending_action': state.pendingAction = event.payload; break
        case 'pending_prompt': state.pendingPrompt = event.payload; break
        case 'turn.end':       state.end = event; state.status = null; break
        case 'error':          state.error = event; state.status = null; break
    }
    return state
}
```

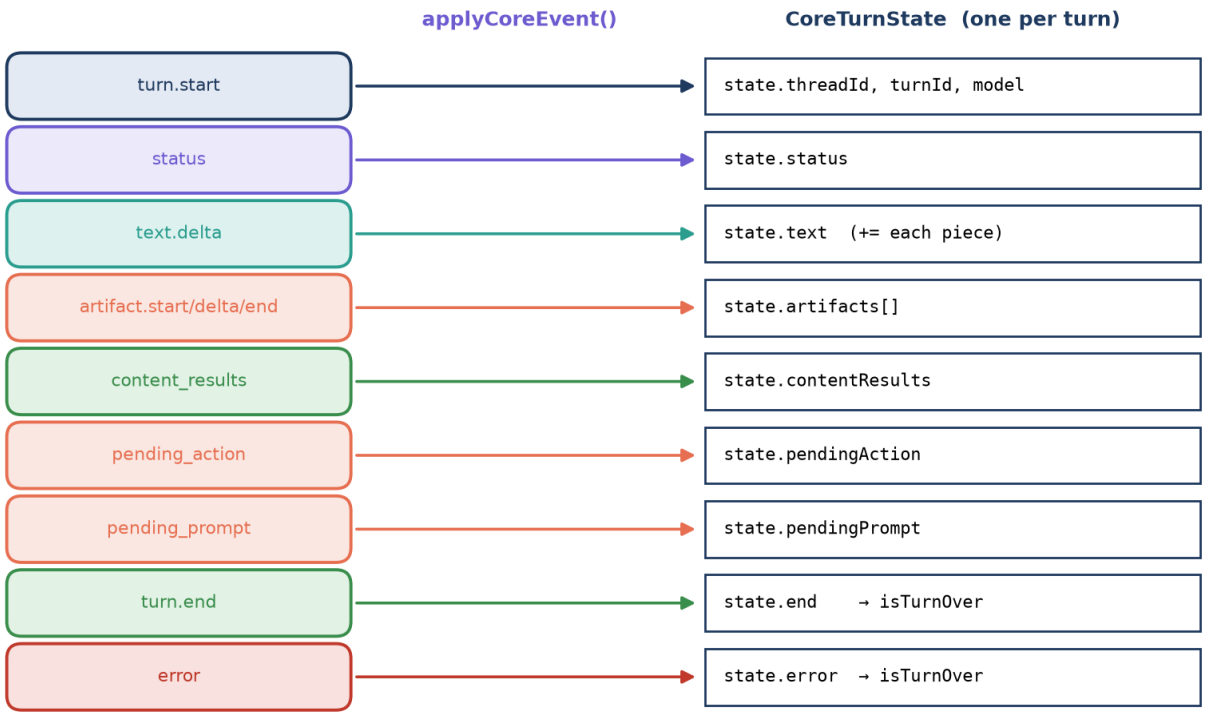


Figure 4.3 — applyCoreEvent(): which event writes which field of the turn's state. turn.end or error makes isTurnOver() true, and later events are ignored.

`sendCoreTurn()` passes an `onEvent` callback that copies the growing text, status and artifacts into `coreLive`, which the Scout window paints. Note it also saves `coreLive.turnId` from `turn.start` — Stop needs it (Chapter 8).

Browser error code	Meaning
`network`	The connection broke while reading
`idle_timeout`	No bytes for 60 s (the server's heartbeat every 10 s should prevent this)
`stream_closed`	The stream ended without a turn.end or error
`stream_overflow`	One event grew past 1 MB

Chapter summary

- ✓ streamTurn() POSTs with fetch() and Accept: text/event-stream; a non-200 is read as a JSON error.
- ✓ consumeCoreStream() races each read against an idle timer and the abort signal.
- ✓ SseParser keeps partial lines, ignores ':' comments and dispatches on a blank line.
- ✓ applyCoreEvent() builds the turn's state; coreLive shows it; turn.start gives the turn_id Stop needs.

Chapter 5

How a Turn Can End

Four endings, and one of them is not really an ending.

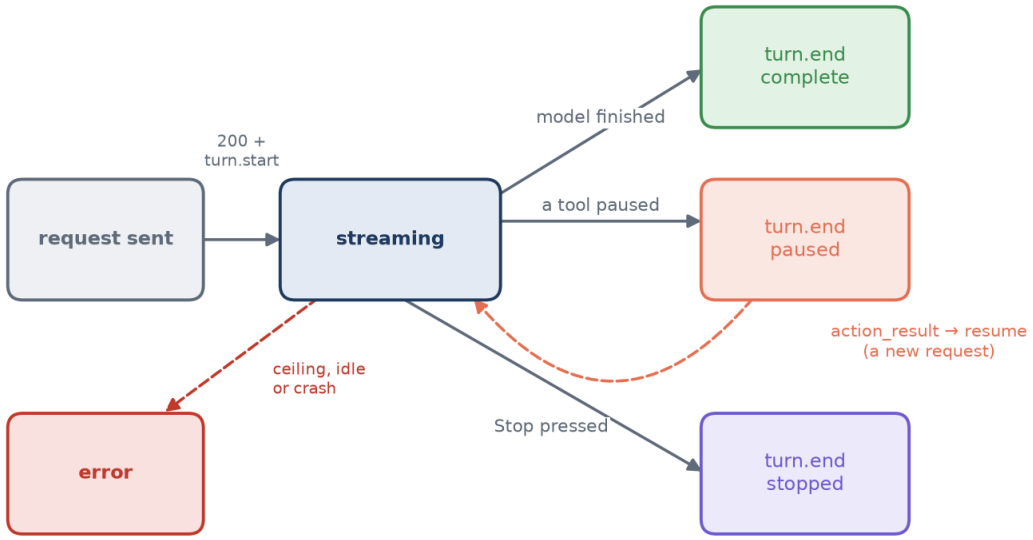


Figure 5.1 — UML state diagram of one turn's stream.

Ending	What the teacher sees	Next
`turn.end {status: "complete"}`	The full answer	Nothing; the teacher can ask again
`turn.end {status: "paused"}`	An action running, an approval card, or a question	The browser sends an action_result as a **new request**, and the same turn continues
`turn.end {status: "stopped"}`	The part shown so far, marked stopped	The thread stays usable
`error`	A friendly error with Try again	Retry, if retryable

In simple words

Paused is like a waiter who asks "spicy or mild?". The order is not finished, and not cancelled — it is waiting for your answer. Your answer goes back as a new message, and the same order continues.

Chapter summary

- ✓ Every stream ends with exactly one of: turn.end complete, paused, stopped — or error.
- ✓ Paused is not finished: the next request carries the answer and continues the same tool call.

Chapter 6

Pause: When Scout Must Wait

Some steps can only happen in the browser, or only the teacher can decide.

6.1 Why a turn pauses

The service **proposes**; the browser **acts**. Clicking a tab, opening a dialog, assigning work — those run in the browser through the executor, with the teacher's own login. And some questions only the teacher can answer ("which grade?"). So when the model calls `page_action` or `ask_user`, the tool does not finish the job. It returns a **pause**.

6.2 What a pause carries

rag/lambdas/ai_assistant/src/core/tools/base.py — *shortened*

```
@dataclass
class PendingAction:
    """Contract §4.1 `pending_action`. `id` is the paused toolUse id."""
    id: str
    capability: str
    args: Dict[str, Any] = field(default_factory=dict)
    state_version: Optional[int] = None
    resume_instruction: Optional[str] = None      # not on the wire
    def to_wire(self) -> Dict[str, Any]:
        wire = {"id": self.id, "capability": self.capability, "args": self.args}
        if self.state_version is not None:
            wire["stateVersion"] = self.state_version
        return wire
@dataclass
class PendingPrompt:
    """Contract §4.5 `pending_prompt`. `id` is the paused toolUse id."""
    id: str
    question: str
    options: List[str] = field(default_factory=list)
    allow_free_text: bool = True
    field: Optional[Dict[str, Any]] = None
    resume_instruction: Optional[str] = None      # not on the wire
    resume: Optional[Dict[str, Any]] = None       # not on the wire: a direct continuation
```

The `id` is the model's own **toolUse id**. That is the key that later lets the answer go back to exactly the tool call that asked for it.

6.3 The loop pauses

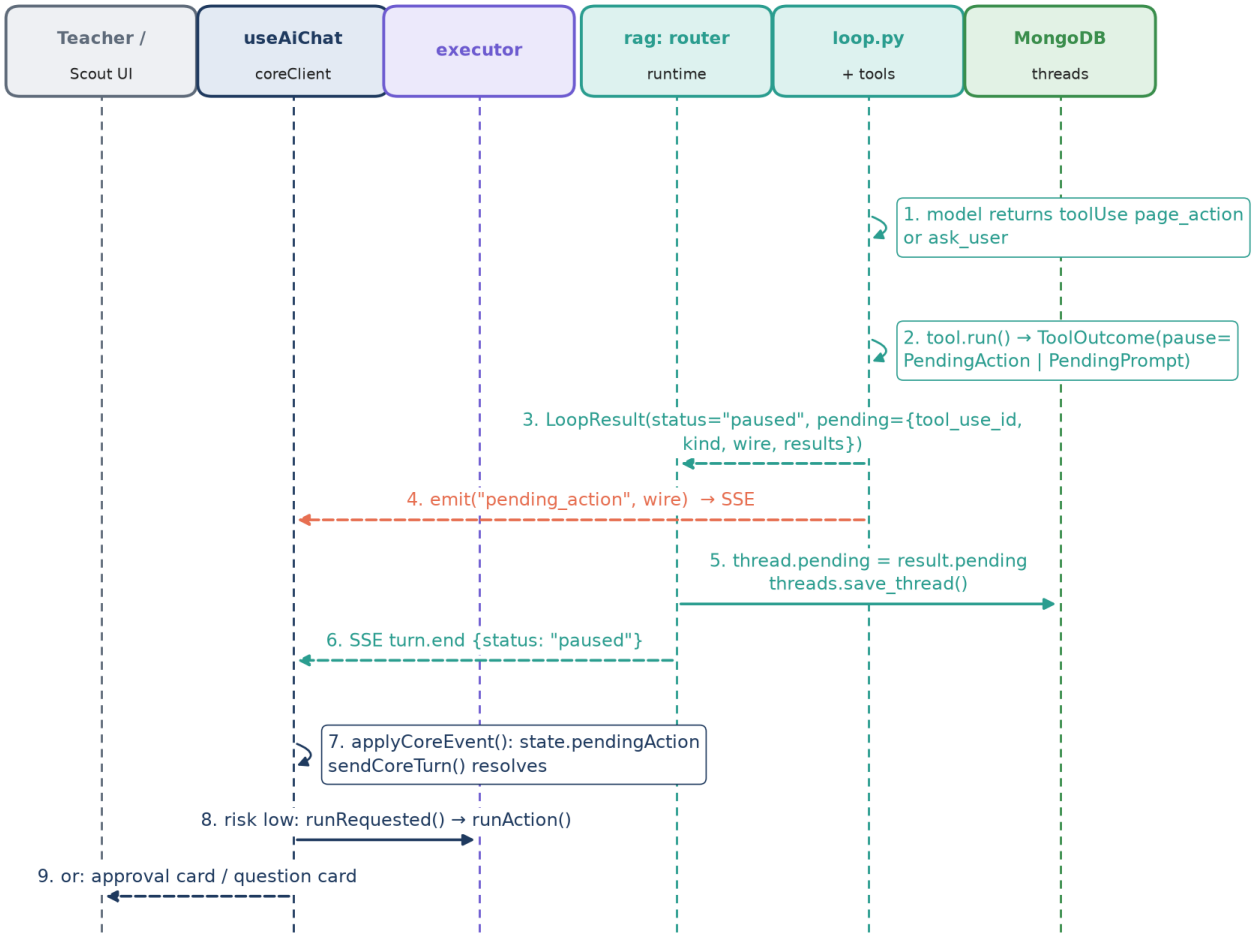


Figure 6.1 — UML sequence diagram of a pause, from the tool to the browser.

rag/lambdas/ai_assistant/src/core/loop.py — _run_loop() — *shortened*

```
for use, outcome in zip(uses, outcomes):
    …
    if outcome.pause is not None:
        if pause is None:
            pause, pause_use = outcome.pause, use
            continue
        outcome = ToolOutcome.error(
            "Only one action or question can wait on the teacher at a time. "
            "This one was not shown; ask again after they respond."
        )
    blocks.append(tool_result_block(use.get("toolUseId", ""), _outcome_text(outcome),
                                    outcome.is_error))
if pause is not None and pause_use is not None:
    if not pause.id:
        pause.id = pause_use.get("toolUseId", "")
    wire = pause.to_wire()
    result.status = "paused"
    result.pause = pause
    result.pending = {
        "tool_use_id": pause_use.get("toolUseId"),
        "tool": pause_use.get("name"),
        "kind": "action" if isinstance(pause, PendingAction) else "prompt",
        "wire": wire,
        "resume_instruction": getattr(pause, "resume_instruction", None),
        "resume": getattr(pause, "resume", None),
        "results": blocks,                        # the other tools' results, kept for the resume
```

rag/lambdas/ai_assistant/src/core/loop.py — _run_loop() — *shortened*

```
    }
    await emit(_pause_event(pause), wire)         # SSE: pending_action or pending_prompt
    return result
```

- **Only one pause at a time.** If the model calls two pausing tools together, the second gets an error result telling it to ask again later.
- **Sibling results are kept.** If other tools ran in the same step, their results are saved in `results` and sent back together with the answer, so the model sees a complete step.
- Back in `_run_turn()`, `thread.pending = result.pending` is saved to MongoDB and `turn.end {status: "paused"}` is sent. The stream closes. **Nothing is waiting in memory** — the next request may even reach a different Lambda environment.

This is exactly what is saved at that moment, using the pacing example from the developer guide:

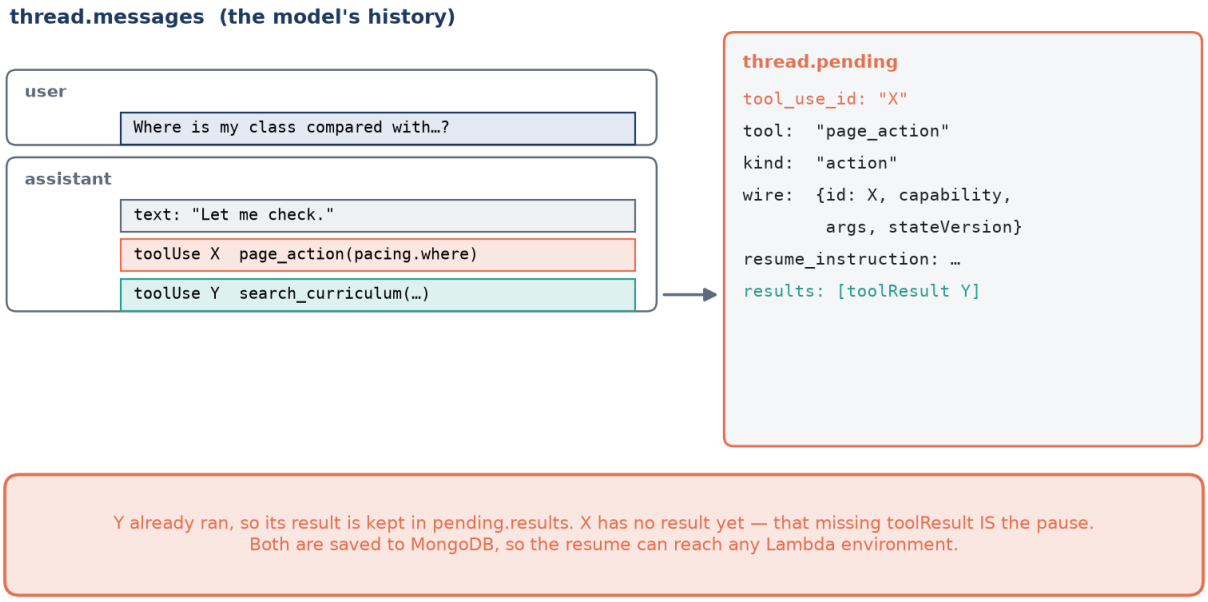


Figure 6.2 — The model's history and the pending record at the moment of a pause.

6.4 Three kinds of pause in the browser

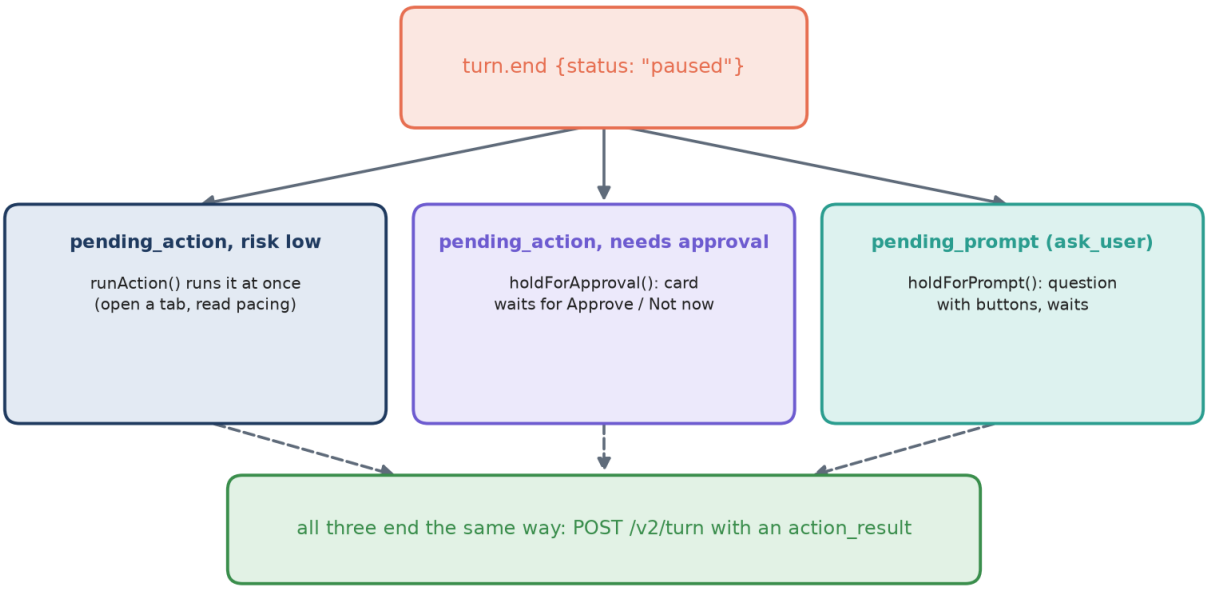


Figure 6.3 — What the browser does with each kind of pause.

swo-3.0/frontend-v3/ai/composables/useAiChat.ts — *shortened*

```
const pending = data.pending_action
if (!pending) break
…
const outcome = await actions.runRequested(pending)          // executor: gates, run(), verify()

if (outcome.status === 'needs_confirmation' && outcome.confirm) {
    holdForApproval(pending.id, outcome.confirm, data, requestThreadId)   // show the card, wait
    return { data, held: true, acted }
}
…
data = await sendTurn(withTurn({
    message: '',
    thread_id: data.thread_id ?? requestThreadId,
    action_result: actions.resultFor(pending.id, pending.capability, outcome),
    …
}), await getAuth())
continue                                                      // the resumed turn may pause again
```

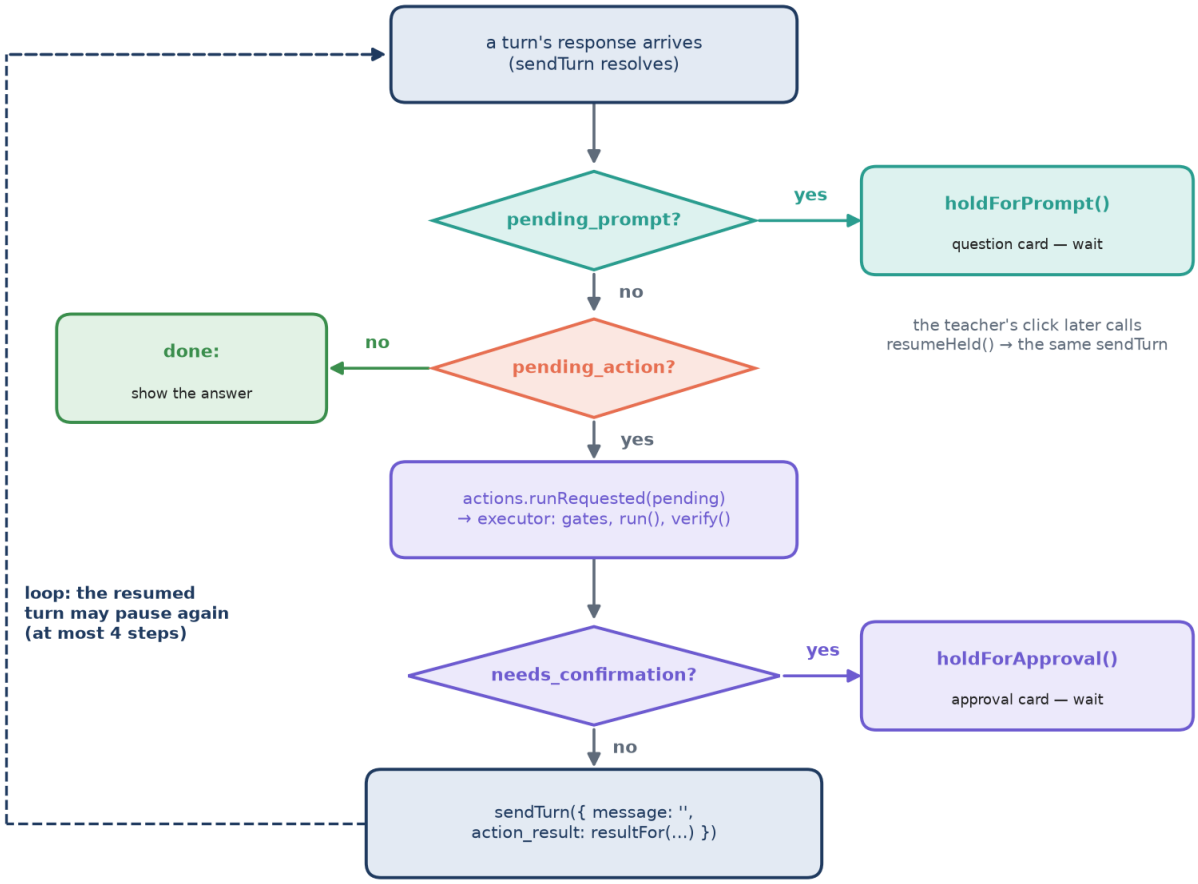


Figure 6.4 — The browser's loop after each response: hold for the teacher, or run the action and resume, until the turn completes.

A low-risk action runs at once and the turn is resumed automatically, in a loop of at most `MAX_ACTION_STEPS = 4` steps. An action that needs approval, or a question, is **held**: the card stays on screen and nothing is sent until the teacher clicks.

Chapter summary

- ✓ page_action and ask_user don't finish the job; they return a PendingAction or PendingPrompt.
- ✓ The pause carries the model's toolUse id, so the answer can go back to that exact call.
- ✓ The loop saves pending (with sibling results) in MongoDB, emits pending_*, and ends with turn.end paused.
- ✓ The browser runs low-risk actions at once, or holds an approval card or question until the teacher clicks.

Chapter 7

Resume: Carrying On After the Pause

A new request, the same thread, the same tool call.

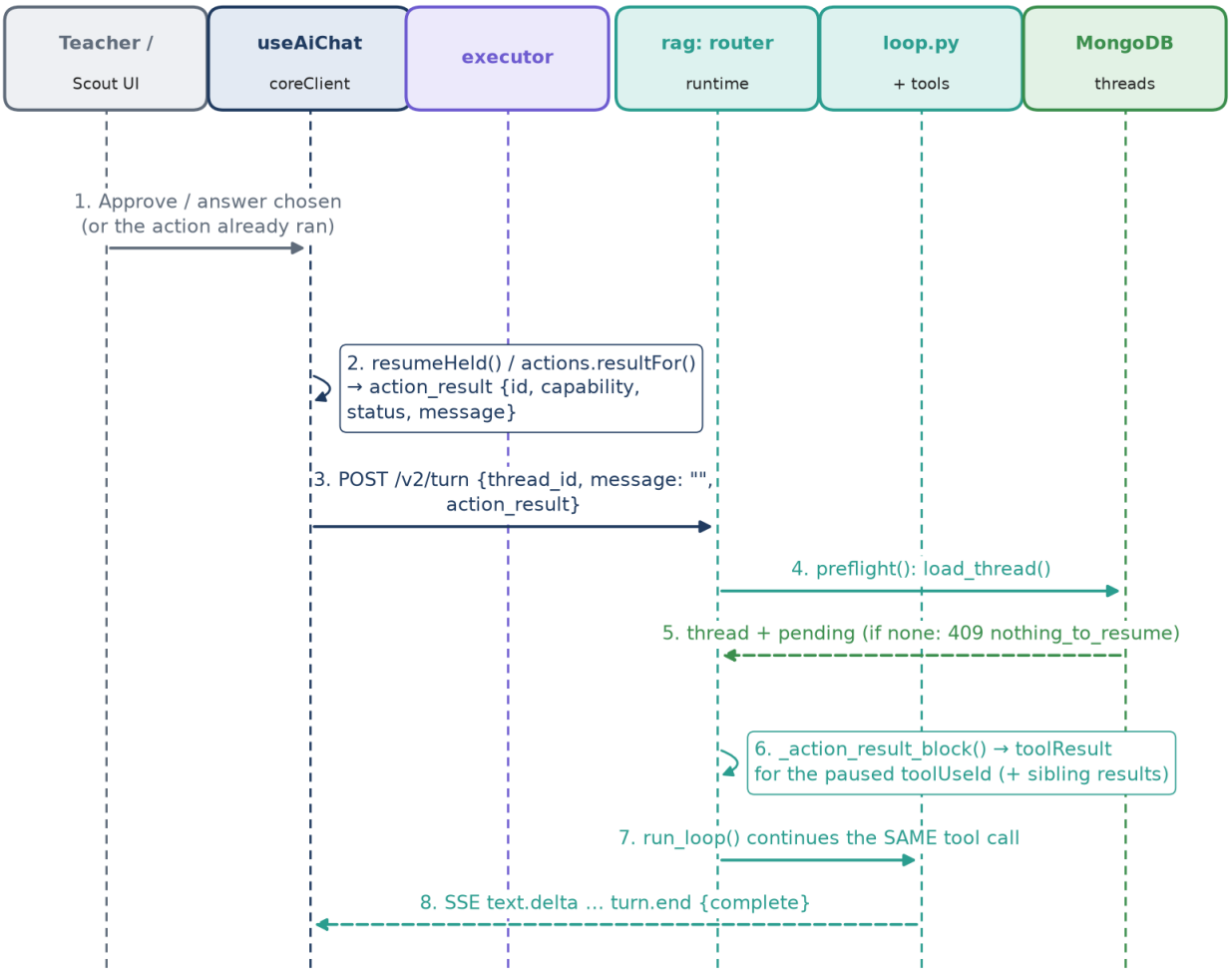


Figure 7.1 — UML sequence diagram of a resume.

7.1 The browser sends the answer

Every kind of pause ends the same way: a new POST to `/v2/turn` with an empty message and an `action_result`. For a question, the answer is the `message` and `capability` is empty:

swo-3.0/frontend-v3/ai/composables/useAiChat.ts

```
async function answerHeldPrompt(answer: string) {
    const held = store.heldApproval
    const message = heldMessage()
    if (!held || held.kind !== 'prompt' || !message?.prompt) return

    message.prompt.answer = answer
    await resumeHeld(
        {
            id: message.prompt.promptId,       // the paused toolUse id
            capability: '',                     // empty: this answers a question
            status: 'ok',
            message: answer,
            stateVersion: currentStateVersion(),
        },
```

swo-3.0/frontend-v3/ai/composables/useAiChat.ts

```
        'answered',
    )
}
```

7.2 The server checks there is something to resume

rag/lambdas/ai_assistant/src/core/runtime.py — preflight()

```
if getattr(request, "action_result", None) and (thread is None or not thread.pending):
    raise PreflightError(409, "nothing_to_resume")
```

7.3 The answer becomes the tool's result

The model asked with a tool call, so it must get a **tool result** for that same call. `_action_result_block()` writes one, using the saved `tool_use_id`:

rag/lambdas/ai_assistant/src/core/runtime.py — *shortened*

```
def _action_result_block(pending, action_result, ask="", transcript=None):
    tool_use_id = pending.get("tool_use_id") or action_result.get("id") or ""
    capability = str(action_result.get("capability") or "").strip()
    message = str(action_result.get("message") or "").strip()
    status = str(action_result.get("status") or "ok")
    if not capability or pending.get("kind") == "prompt":
        answer = f"The teacher answered: {message or '(no answer)'}."
        instruction = str(pending.get("resume_instruction") or "").strip()
        return tool_result_block(tool_use_id, f"{answer} {instruction}" if instruction else answer)
    parts = [f"The browser's result for {capability}: {status}."]
    if message:
        parts.append(message)
    …                                   # completion, resolution, page state before → after
    return tool_result_block(tool_use_id, " ".join(parts), is_error=(status == "error"))
```

Then `_run_turn()` appends the saved sibling results plus this block as the next user message, clears `thread.pending`, and runs the loop again. From the model's point of view, its tool call simply returned.

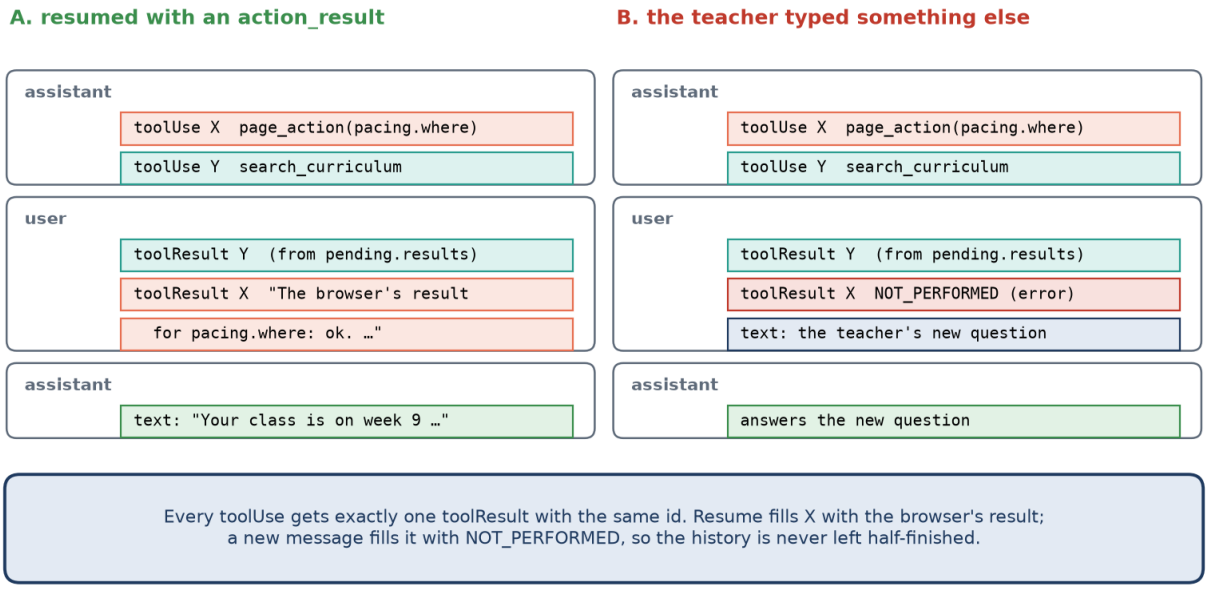
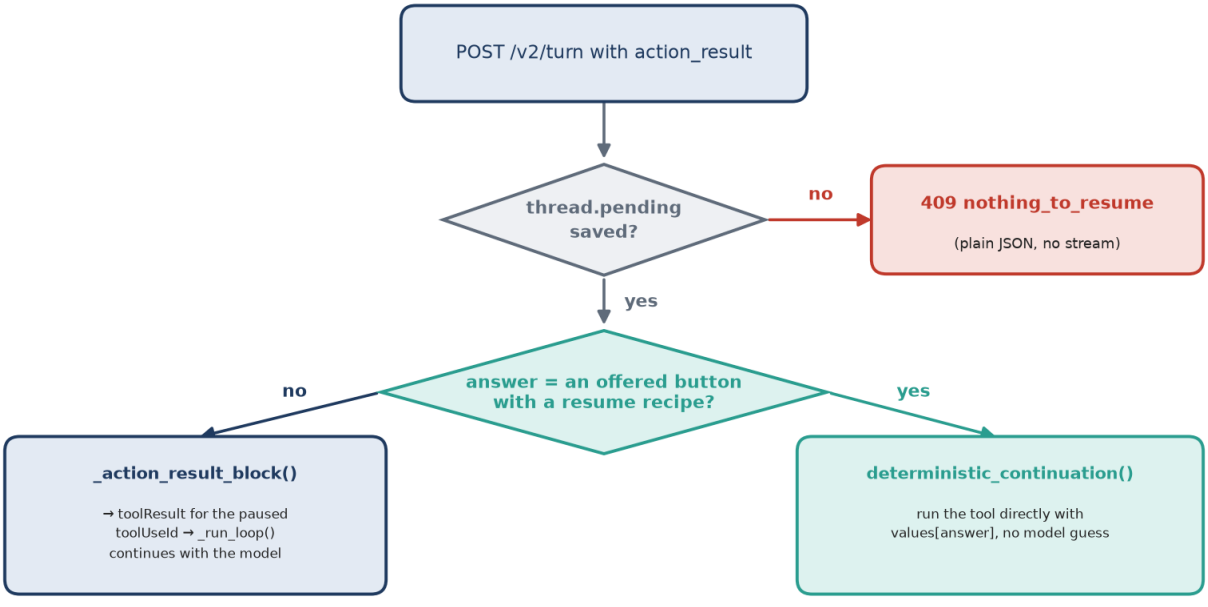


Figure 7.2 — The model's history after the pause is closed: by the browser's result (A), or by NOT_PERFORMED when the teacher typed something else (B).

Why it is built this way

Some answers skip the model. A `PendingPrompt` can carry a `resume` recipe: if the teacher clicks exactly one of the offered buttons, the runtime runs the tool directly with that value (`deterministic_continuation()`). For example, after "Grade 2" the leveling tool runs at once, so the model cannot claim a rewrite it never did.



Example: after "Which grade?" the teacher clicks "Grade 2" → level_article runs at once, so a rewrite cannot be claimed without being done.

Figure 7.3 — How the server routes an action_result: refuse, run the tool directly, or hand the result back to the model.

7.4 If the teacher types something else instead

A teacher may ignore the card and type a new question. The paused call still needs a result, or the conversation would be broken. So the runtime closes it with a clear note:

rag/lambdas/ai_assistant/src/core/runtime.py — *two excerpts*

```
NOT_PERFORMED = (
    "Not performed: the teacher sent a new message instead of answering or approving this, "
    "so nothing happened."
)
…
if thread.pending:
    blocks.extend(thread.pending.get("results") or [])
    blocks.append(tool_result_block(thread.pending.get("tool_use_id", ""), NOT_PERFORMED,
                                    is_error=True))
    thread.pending = None
```

Chapter summary

- ✓ Resume is a new POST /v2/turn with an empty message and an action_result.
- ✓ preflight() refuses a resume with nothing pending (409 nothing_to_resume).
- ✓ _action_result_block() turns the answer into a toolResult for the paused toolUse id; the loop continues.
- ✓ A clicked button can run the tool directly; a new typed message closes the pause as NOT_PERFORMED.

Chapter 8

Stop: Ending a Turn Early

Closing the stream is not enough — the service must be told.

8.1 Why Stop needs two steps

When the teacher presses Stop, the browser closes the stream. But on Lambda, a closed connection may never reach the environment doing the work, and the service deliberately keeps a turn running when its client leaves (so a reload still shows the answer). If we only closed the stream, the saved thread would contain the full answer the teacher stopped. So Stop does two things:

- **Abort** the fetch — this ends the turn *here*, in the browser.
- **POST** `/v2/turn/cancel` — this ends it *there*, on the service, through MongoDB.

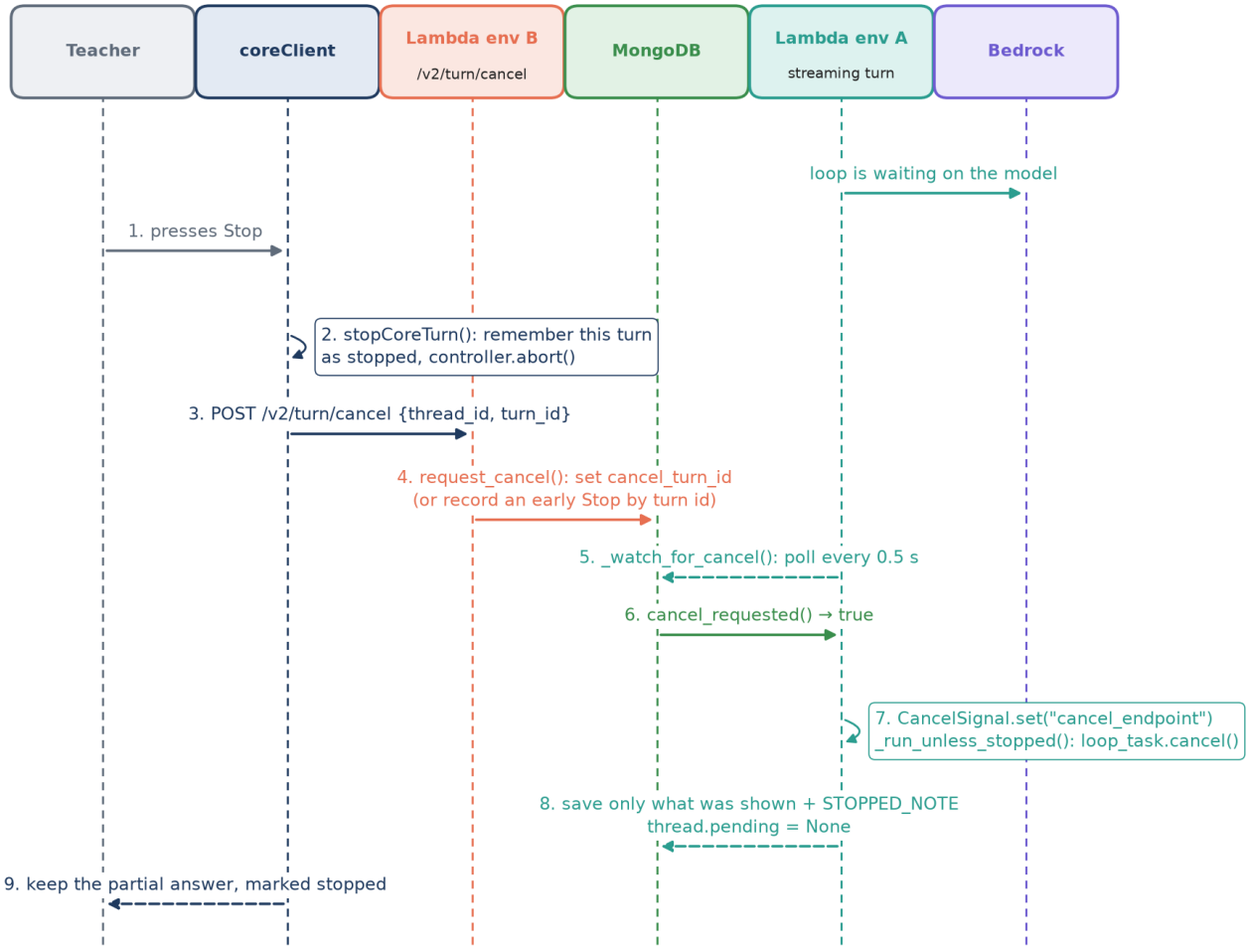


Figure 8.1 — UML sequence diagram of Stop. The cancel may land on a different Lambda environment from the one streaming.

8.2 The browser side

swo-3.0/frontend-v3/ai/components/AiComposer.vue

```
function stopTurn() {
  if (!flow.stopIntent()) stopCoreTurn()
}
```

swo-3.0/frontend-v3/ai/services/coreClient.ts

```
export function stopCoreTurn(): void {
    const controller = liveCoreController()
    if (!controller) return
    stoppedTurns.add(controller)            // a Stop, not a walk-away
    const { threadId, turnId } = coreLive   // turnId came from turn.start
    const auth = turnAuth.get(controller)
    controller.abort()
    if (auth && threadId && turnId) {
        void aiFetch(`${coreRoot()}/v2/turn/cancel`, {
            method: 'POST',
            auth,
            body: { thread_id: threadId, turn_id: turnId, user_id: auth.userId },
            isSuccess: () => true,
        }).catch(() => undefined)
    }
}
```

When the abort reaches `sendCoreTurn()`, it checks `stoppedTurns`. A Stop turns the partial state into an answer marked stopped, so what the teacher already read stays on screen. Any other abort (leaving the conversation) stays an abort and shows nothing.

swo-3.0/frontend-v3/ai/services/coreClient.ts — sendCoreTurn() *— shortened*

```
} catch (error: unknown) {
    …
    if (stoppedTurns.has(controller) && error instanceof Error && error.name === 'AbortError') {
        const partial = (error as CoreAbortError).state
        return coreChatResponse(partial ?? initialTurnState(), body.thread_id, { stopped: true })
    }
    …
}
```

8.3 The service records the Stop in MongoDB

rag/lambdas/ai_assistant/src/core/threads.py *— shortened*

```
def request_cancel(thread_id: str, user_id: str, turn_id: str) -> str:
    doc = _collection().find_one({"_id": thread_id},
                                 {"user_id": 1, "running_turn_id": 1, "transcript.turn_id": 1})
    if doc is not None and str(doc.get("user_id")) != str(user_id):
        return "not_found"
    if doc is not None:
        result = _collection().update_one(
            {"_id": thread_id, "user_id": user_id, "running_turn_id": turn_id},
            {"$set": {"cancel_turn_id": turn_id, "cancel_at": _utcnow()}},
        )
        if result.matched_count:
            return "cancelled"
        if any(row.get("turn_id") == turn_id for row in doc.get("transcript") or []):
            return "not_running"                  # already finished: a no-op
    # Not running yet (a first turn has no document at all): record the Stop by turn id.
    _cancels().update_one({"_id": turn_id},
                          {"$set": {"thread_id": thread_id, "user_id": user_id, "at": _utcnow()}},
                          upsert=True)
```

rag/lambdas/ai_assistant/src/core/threads.py — *shortened*

```
    return "cancelled"
```

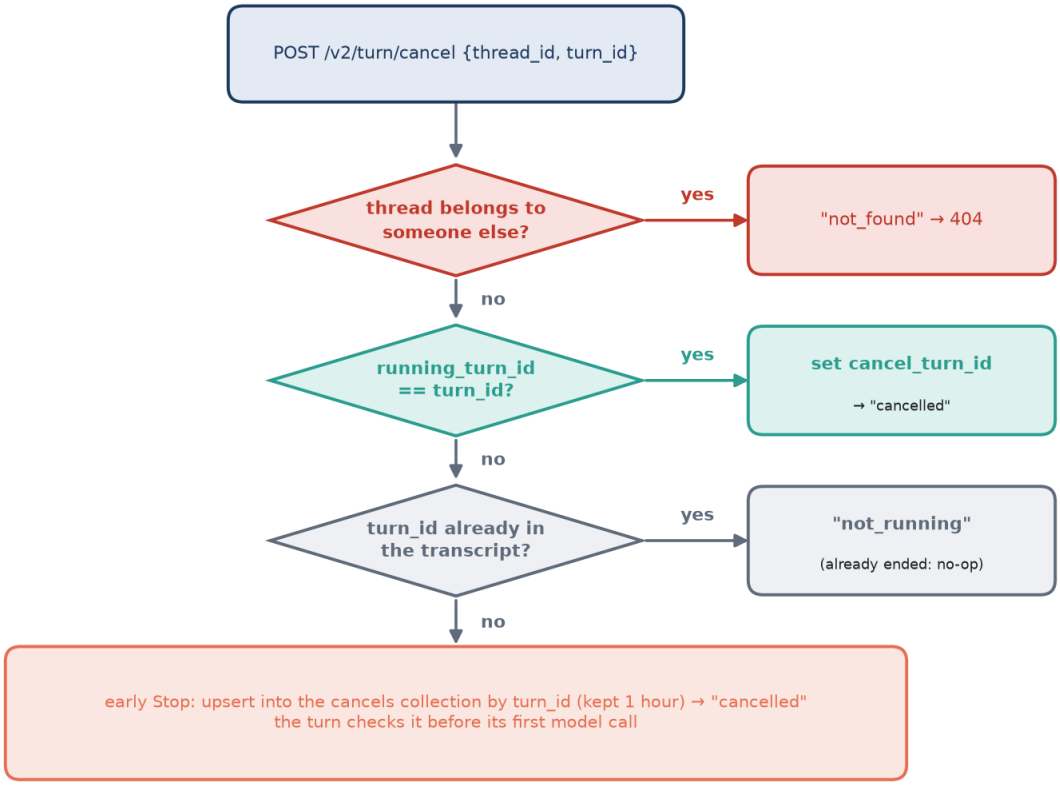


Figure 8.2 — request_cancel(): the four possible answers, as a flowchart.

The early-Stop collection keeps entries for one hour. It covers a Stop pressed so quickly that the turn has not yet marked itself as running.

8.4 The running turn notices and stops

rag/lambdas/ai_assistant/src/core/runtime.py — *shortened*

```
async def _watch_for_cancel(thread_id, turn_id, user_id, cancel: CancelSignal,
                            poll_s: float) -> None:
    """Poll the store for a Stop recorded by any process: at once, then every `poll_s`."""
    while not cancel.event.is_set():
        if await _cancel_seen(thread_id, turn_id, user_id):
            cancel.set("cancel_endpoint")
            return
        await asyncio.sleep(poll_s)               # 0.5 s
async def _run_unless_stopped(loop_coro, cancel, thread_id, turn_id, poll_s, user_id=""):
    if cancel.event.is_set() or await _cancel_seen(thread_id, turn_id, user_id):
        cancel.set("cancel_endpoint")             # stopped before the first model call
        loop_coro.close()
        return None
    loop_task = asyncio.create_task(loop_coro)
    stop_wait = asyncio.create_task(cancel.event.wait())
    poller = asyncio.create_task(_watch_for_cancel(thread_id, turn_id, user_id, cancel, poll_s))
    try:
        await asyncio.wait({loop_task, stop_wait}, return_when=asyncio.FIRST_COMPLETED)
        if loop_task.done():
            return loop_task.result()
        loop_task.cancel()                        # also cancels the model call it awaits
        …
        return None
```

rag/lambdas/ai_assistant/src/core/runtime.py — *shortened*

```
    finally:
        if not loop_task.done():
            loop_task.cancel()
        stop_wait.cancel()
        poller.cancel()
```

The race in that code, drawn on a timeline:

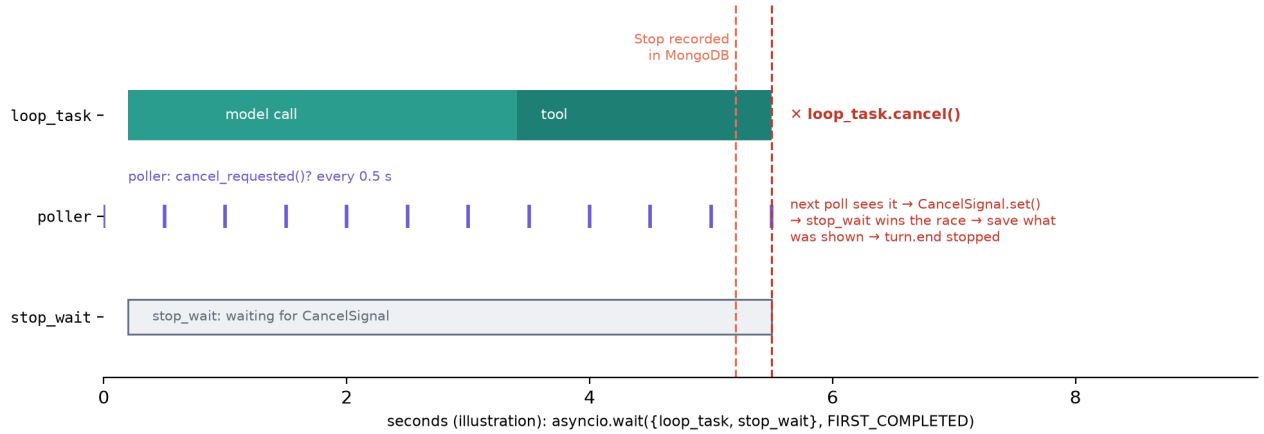


Figure 8.3 — Three tasks run together. The loop works, the poller checks MongoDB every 0.5 s, and stop_wait waits. Whichever of loop_task or stop_wait finishes first decides the turn.

8.5 What is saved after a Stop

rag/lambdas/ai_assistant/src/core/runtime.py — _run_turn() — *shortened*

```
if result is None:
    # Stop: keep only what the teacher was shown.
    outcome = "stopped"
    shown = collector.text.strip()
    thread.messages = thread.messages[:base_len]
    thread.messages.append({"role": "assistant", "content": [
        {"text": f"{shown}\n\n{STOPPED_NOTE}" if shown else STOPPED_NOTE}]})
    # STOPPED_NOTE = "[The teacher stopped this answer here.]"
    thread.pending = None
    …
    await asyncio.to_thread(threads.save_thread, thread)
    await emit("turn.end", {"status": "stopped", "outcome": "stopped",
                            "stopped_by": cancel.reason, …})
    return
```

Why it is built this way

The history keeps **only what the teacher saw**, plus a note. Unfinished tool calls are dropped, so nothing is left dangling, and the next turn's model knows the answer was cut short.

Watch out

Locally, a closed connection is also treated as a Stop (`cancel.set("disconnect")` in `stream_turn()`). On Lambda this is not reliable, which is why the explicit cancel endpoint is the real Stop. Don't test Stop only by closing the tab.

Chapter summary

- ✓ Stop = abort the fetch (here) + POST /v2/turn/cancel (there).
- ✓ request_cancel() marks the turn in MongoDB, or records an early Stop by turn id for one hour.
- ✓ The running turn polls every 0.5 s, then cancels the loop and any model call it is waiting on.
- ✓ Only what was shown is saved, with '[The teacher stopped this answer here.]'; the browser keeps the partial answer.

Chapter 9

Timers, Failures and Debugging

What keeps a stream alive, what ends it, and how to look inside.

9.1 The timers

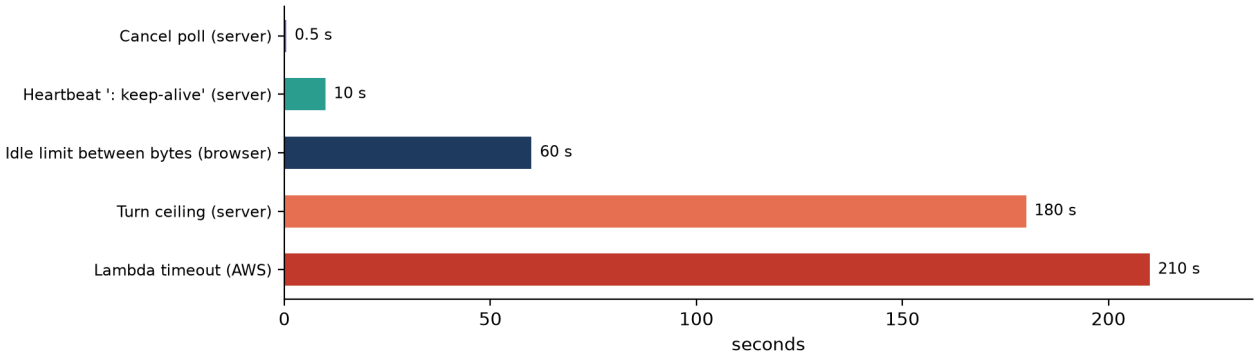


Figure 9.1 — The time limits on a turn, smallest to largest.

Timer	Value	Where	Why
Cancel poll	0.5 s	`CORE_CANCEL_POLL_S`	How quickly a Stop is noticed
Heartbeat	10 s	`CORE_HEARTBEAT_S`	Keeps proxies from closing a quiet line
Browser idle limit	60 s	`CORE_IDLE_TIMEOUT_MS`	A silent stream is treated as dead
Turn ceiling	180 s	`CORE_TURN_CEILING_S`	Ends the turn with its own error and a saved thread
Lambda timeout	210 s	Terraform	Kept 30 s above the ceiling, so Lambda never kills a turn mid-save

9.2 What happens when something fails

Situation	What happens
Bad request, not signed in, thread not yours	JSON error before any stream (400 / 401 / 404 / 409)
Model or tool crashes mid-turn	One `error` event with code `internal`; retryable
Turn runs past 180 s	`error` with code `turn_ceiling`; telemetry still written
Network drops	Browser shows `network`; the server finishes and saves, so reopening the conversation shows the answer
Teacher leaves the conversation	The fetch is aborted; nothing is shown in the new conversation
A cancel arrives after the turn ended	No-op (`not_running`)

9.3 Looking inside

- **Browser:** DevTools → Network → the `/v2/turn` request. Its response holds the raw events. In the console, `window.__aiContextDebug = true` logs what the browser sends.
- **Server logs:** every turn logs its outcome (`complete / paused / stopped`) and timings; `turn.end` carries `timings`, `ttft_ms` and `cost_usd`.
- **Raw stream with curl** (needs a valid token and host; `-N` turns off curl's buffering):

terminal — *example*

```
curl -N -X POST "$AI_CORE_URL/v2/turn?host=$SWO_HOST" \
  -H "Authorization: $TOKEN" -H "Content-Type: application/json" -H "Accept: text/event-stream" \
  -d '{"message": "What is this page?", "user_id": "…"}'
```

Try it yourself

Ask Scout for a lesson pack and press Stop after a few lines. Then reopen the conversation from History: you should see exactly the lines you saw, ending with the stopped note — not the full pack.

Chapter summary

- ✓ Timers: cancel poll 0.5 s, heartbeat 10 s, browser idle 60 s, turn ceiling 180 s, Lambda 210 s.
- ✓ Early failures are JSON; mid-stream failures are one error event; a dropped network still saves the answer.
- ✓ Look at the /v2/turn response in DevTools, turn.end timings, or the raw stream with curl -N.